

UNIVERSIDADE FEDERAL DE ALAGOAS
INSTITUTO DE COMPUTAÇÃO
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO
DISCIPLINA: PROGRAMAÇÃO 2

ERASMO DA SILVA SÁ JUNIOR

RELATÓRIO DA MILESTONE 2

Maceió, Alagoas
2026

1. INTRODUÇÃO

O Jackut é um sistema que mantém uma rede de relacionamentos, nos moldes de outras redes sociais amplamente conhecidas (Facebook, Orkut, Friendster, entre outras), sendo particularmente inspirado no Orkut. A funcionalidade desejada do sistema é descrita por meio de User Stories, que relatam pequenos casos de uso informais, cada um descrevendo uma interação que o sistema deve ser capaz de suportar. As User Stories recebem prioridades definidas pelo cliente interessado e são implementadas seguindo rigorosamente essa ordem.

Este relatório documenta o estado atual de desenvolvimento do projeto Jackut, com foco em três frentes: (i) uma descrição da arquitetura e dos padrões de projeto efetivamente aplicados no código; (ii) uma avaliação crítica do design atual, destacando virtudes e pontos de atenção; e (iii) uma verificação detalhada do alinhamento entre os diagramas UML presentes na pasta docs/ do repositório e a estrutura real do código-fonte, item solicitado explicitamente para esta entrega. De forma complementar, descreve o refatoramento de encapsulamento, herança e polimorfismo realizado de acordo com o feedback recebido referente a Milestone 1, bem como o desenvolvimento das User Stories 05 a 09 (comunidades, novos tipos de relacionamento e exclusão de conta).

O restante do documento está organizado da seguinte forma: a seção 2 apresenta uma visão geral da arquitetura; a seção 3 traz a avaliação crítica do design, com pontos fortes e pontos de atenção; a seção 4 evidencia os diagramas UML solicitados, o nível de alinhamento entre os diagramas UML e o código atual; a seção 5 descreve o refatoramento e o desenvolvimento realizado; a seção 6 apresenta as conclusões.

2. VISÃO GERAL DA ARQUITETURA

2.1 Estrutura de Pastas e Camadas

O código-fonte está organizado sob o pacote-base `br.ufal.ic.jackut`, dividido em camadas com responsabilidades bem delimitadas:

- **exceptions/** — 29 classes de exceção, cada uma representando uma regra de negócio específica (ex.: `LoginOuSenhaInvalidoException`, `AmizadeJaAdicionadaException`, `AutoRecadoException`).
- **models/** — entidades de negócio (`Usuario`, `Comunidade`, `Session`) e as hierarquias polimórficas `Relacionamento` (`Amizade`, `Idolatria`, `Paquera`, `Inimizade`) e `Mensagem` (`Recado`, `MensagemComunidade`), além da fábrica `RelacionamentoFactory`.
- **repository/** — persistência de cada entidade via serialização XML (`java.beans.XMLEncoder/XMLDecoder`), com métodos estáticos `save()`, `load()` e `clear()`.
- **services/** — `UsuarioService`, `AmizadeService`, `RecadoService`, `ComunidadeService` e `SessionService`, cada um orquestrando as regras de negócio de seu domínio.
- **utils/** — a classe `Validador`, com validações reutilizáveis entre serviços (login, senha, sessão, credenciais, existência de usuário).
- **Facade** — ponto único de entrada usado pelos testes de aceitação (`EasyAccept`) e pela classe `Main`, delegando toda a lógica aos `services`.

O fluxo de execução segue a cadeia `Main → Facade → Services → Models → Repository`, com as `Exceptions` atravessando todas as camadas para sinalizar violações de regras de negócio.

2.2 Padrões de Projeto Utilizados

- **Facade** — a classe `Facade` concentra a interface pública do sistema, sem conter lógica de negócio própria; toda chamada é delegada a um `service`.

- **Service Layer** — cada service centraliza a lógica de um domínio (usuários, amizades/relacionamentos, recados, comunidades, sessões), evitando uma classe única e gigante ("God Class").
- **Repository** — a persistência fica isolada em classes dedicadas, permitindo trocar a estratégia de armazenamento sem alterar os services.
- **Factory Method** — `RelacionamentoFactory.criarRelacionamento(tipo, ...)` centraliza a criação dos quatro subtipos de relacionamento a partir de uma string.
- **Herança e Polimorfismo** — `Relacionamento` e `Mensagem` são classes abstratas com métodos `validar()/aplicar()/tipo()` (Relacionamento) sobrescritos pelas subclasses, eliminando condicionais de tipo.
- **Guard Clauses** — a classe `Validador` concentra validações de entrada no início dos métodos, evitando aninhamento profundo de condicionais.

3. AVALIAÇÃO DO PROJETO

3.1 Pontos Fortes

A separação em camadas (services, repository, models, exceptions, utils) é consistente e evita a concentração de regras de negócio em uma única classe.

O tratamento de exceções é, em sua maior parte, granular e específico: o projeto conta com 29 classes de exceção dedicadas a regras de negócio pontuais (ex.: `IdolatriaJaAdicionadaException`, `PaqueraJaAdicionadaException`, `InimizadeJaAdicionadaException`), em vez de depender de `java.lang.Exception` genérica, o que facilita tanto a legibilidade quanto o tratamento correto em cada ponto de chamada.

A hierarquia Relacionamento (com Amizade, Idolatria, Paquera e Inimizade) e a hierarquia Mensagem (com Recado e MensagemComunidade) são bons exemplos de herança aplicada com propósito real: os métodos `validar()` e `aplicar()` são sobrescritos por cada subtipo, eliminando a necessidade de condicionais do tipo `"if (tipo.equals(...))"` espalhados pelo código de negócio. A criação desses objetos fica centralizada em `RelacionamentoFactory`, mantendo o `AmizadeService` livre de lógica de instanciação.

Há cuidados pontuais de encapsulamento que vão além do básico: `Comunidade.getMembros()` retorna uma lista imutável (`Collections.unmodifiableList`), e `Usuario.getAtributos()` retorna uma cópia defensiva do mapa interno, práticas que evitam que código externo corrompa o estado interno dos objetos por referência.

Por fim, a Facade se mantém enxuta e sem regra de negócio própria, delegando integralmente aos services.

3.2 Pontos de Atenção

A generalização para Relacionamento e Mensagem foi aplicada de forma consistente na camada de modelo (models/), mas não se propagou para as camadas de serviço e repositório: o sistema continua utilizando `AmizadeService/AmizadeRepository`, que hoje gerenciam todos os quatro tipos de relacionamento, não apenas amizades, e `RecadoService/RecadoRepository`, que tratam

apenas o subtipo Recado (mensagens entre comunidades são tratadas separadamente, dentro de ComunidadeService). Essa nomenclatura herdada de uma fase anterior do projeto, em que só existiam amigos e recados, é a causa direta da divergência entre os diagramas UML e o código, detalhada na seção 4.

Além disso, todas as classes de modelo (Usuario, Relacionamento e suas subclasses, Mensagem e suas subclasses, Comunidade, Session) ainda expõem um par get/set público para praticamente todo atributo interno, incluindo casos em que a regra de negócio nunca chama o setter correspondente fora do próprio construtor. Esse é exatamente o padrão de "boilerplate de JavaBean" já apontado em feedback anterior sobre o projeto: os setters existem porque o mecanismo de persistência escolhido os exige, não porque o domínio realmente precise desse grau de mutabilidade externa.

4. DIAGRAMAÇÃO UML DO PROJETO

4.1 Diagrama de Classes

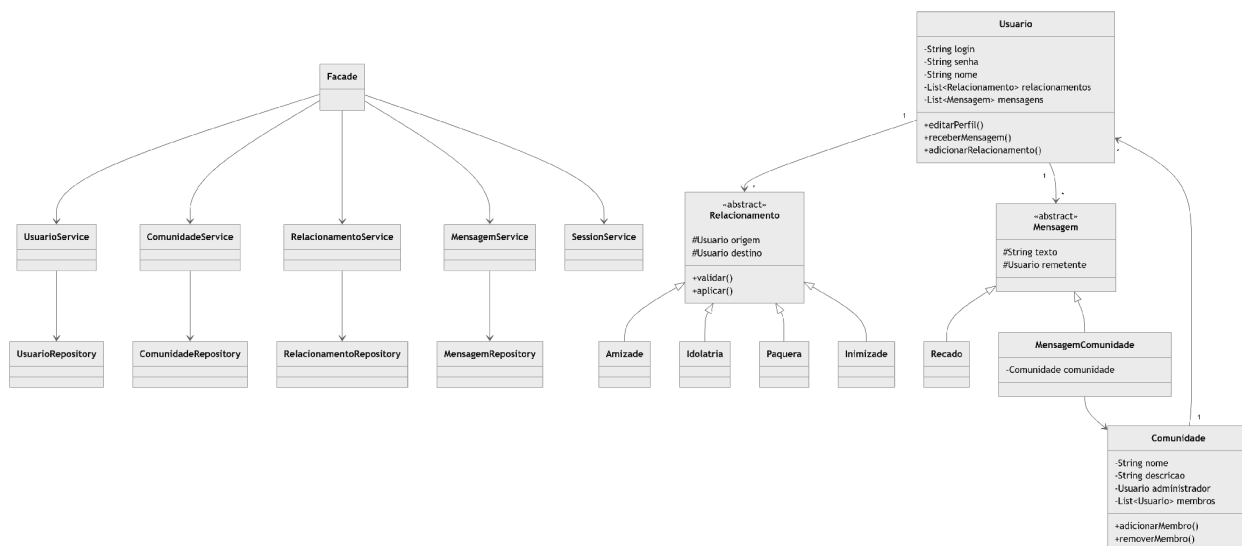


Ilustração 1 — Diagrama de Classes presente em docs/Diagrama de Classes.png

4.2 Diagrama de Pacotes

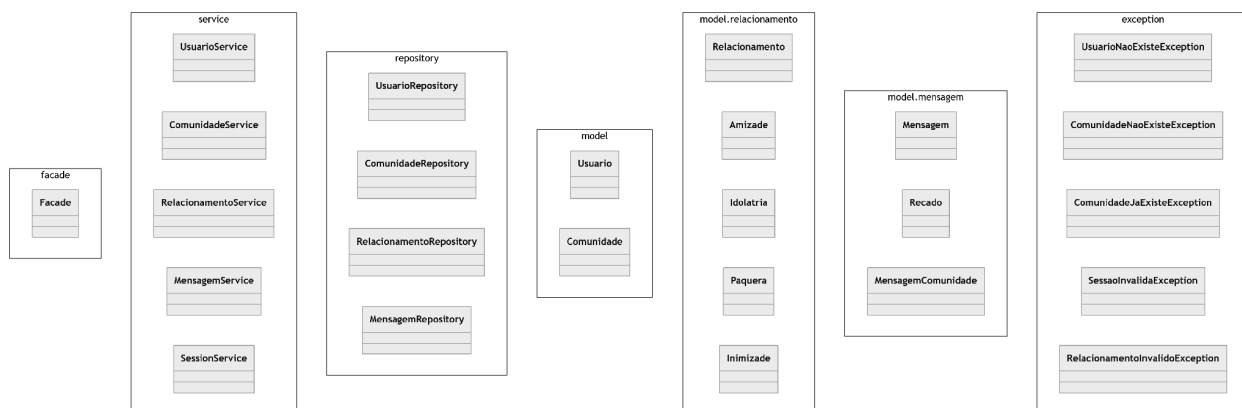


Ilustração 2 — Diagrama de Pacotes presente em docs/Diagrama de Pacotes.png

5. REFATORAMENTO E DESENVOLVIMENTO REALIZADO

Esta seção descreve o refatoramento de encapsulamento, herança e polimorfismo, além do desenvolvimento das User Stories 05 a 09 (Épicos 3, 4 e 5), com base na comparação direta entre o estado anterior e o estado atual do código.

5.1 Refatoração de Encapsulamento e Tratamento de Exceções

Antes da refatoração, Amizade era uma classe concreta e independente, com seus próprios campos solicitante, solicitado e aceito, e o AmizadeService mantinha uma List<Amizade> instanciada diretamente com new Amizade(meuLogin, amigoLogin, false). Esse desenho funcionava para um único tipo de relacionamento, mas não se generalizava para os novos tipos exigidos pela US08 (idolatria, paquera, inimizado) sem duplicar toda a lógica de criação e busca em três outras classes praticamente idênticas.

A refatoração extraiu esses três campos para a nova classe abstrata Relacionamento, da qual Amizade passou a herdar, mantendo apenas a lógica específica do tipo: tipo() retornando "Amizade", validar() lançando AutoAmizadeException quando solicitante e solicitado são o mesmo usuário, e aplicar() marcando o relacionamento como aceito. O AmizadeService foi adaptado para trabalhar com List<Relacionamento> em vez de List<Amizade>, e a criação de novos relacionamentos passou a ser feita por RelacionamentoFactory.criarRelacionamento(tipo, solicitante, solicitado), que decide a subclasse concreta a partir de uma string.

Em paralelo, a maior parte das assinaturas de métodos na Facade e nos services passou a declarar exceções específicas em vez de Exception genérica, e novas exceções de negócio foram criadas para cada nova regra (RelacionamentoInvalidoException, RelacionamentoJaExisteException, TipoRelacionamentoInvalidoException, IdolatriaJaAdicionadaException, PaqueraJaAdicionadaException, InimizadeJaAdicionadaException, entre outras).

5.2 Hierarquia de Relacionamentos: Herança e Polimorfismo (US08)

A US08 exige suportar quatro tipos de relacionamento: Amizade, Idolatria, Paquera e Inimizade. Cada um com validações próprias, sem recorrer a condicionais de tipo espalhados pelo código. A solução implementada usa Relacionamento como superclasse abstrata com os métodos validar(), aplicar() e tipo(), sobrescritos por cada subclasse: Idolatria e Inimizade lançam AutolIdolatriaException/AutoInimizadeException quando o usuário tenta se autorrelacionar; Paquera segue a mesma regra com AutoPaqueraException.

O AmizadeService.adicionarRelacionamento(sessionId, tipo, alvo, sessionService) generaliza o fluxo para os quatro tipos: valida sessão e usuário-alvo, verifica reciprocidade de inimizade (verificarInimigo), cria o relacionamento via RelacionamentoFactory, chama validar() e aplicar() polimorficamente e, por fim, lança a exceção de "já adicionado" específica do tipo antes de persistir. Um detalhe de comportamento implementado na Facade.adicionarPaquera() merece destaque: quando duas pessoas se "paqueram" mutuamente, o sistema detecta a reciprocidade percorrendo a lista de relacionamentos do tipo Paquera e envia automaticamente um recado de notificação, gerado pelo próprio sistema, com remetente "Jackut" para cada uma das partes.

5.3 Comunidades (US05, US06 e US07)

A classe Comunidade foi criada com nome, descricao, administrador e a lista de membros (o criador é automaticamente adicionado como membro no construtor, atendendo ao critério de aceitação da US05 de que "o criador torna-se administrador"). ComunidadeService.criarComunidade() impede nomes duplicados, lançando ComunidadeJaExisteException; participarComunidade() (US06) impede que um usuário entre duas vezes na mesma comunidade, lançando UsuarioJaPertenceException.

Para a US07 (mensagens em comunidades), cada Comunidade mantém sua própria lista de MensagemComunidade, mas a leitura é implementada da seguinte forma: um mapa mensagensLidas guarda, por usuário, o índice da próxima mensagem não lida daquela comunidade, e lerMensagem() avança esse índice a cada leitura — um mecanismo equivalente a um ponteiro de leitura por membro, evitando que

mensagens já lidas sejam reapresentadas. Um segundo mapa, `ordemEntrada`, registra a ordem de entrada de cada membro em cada comunidade, usado por `getComunidades()` para listar as comunidades de um usuário na ordem em que ele efetivamente entrou nelas.

5.4 Exclusão de Conta (US09)

A US09 exige que a remoção de um usuário propague para relacionamentos, mensagens, participação em comunidades e, por fim, para a própria conta. Isso foi implementado na `Facade.removerUsuario(id)`, que obtém o usuário da sessão, valida a sessão e então chama, em sequência, `amizadeService.removerUsuario(login)` (remove todos os relacionamentos em que o usuário aparece como solicitante ou solicitado), `recadoService.removerUsuario(login)` (remove recados enviados ou recebidos), `comunidadeService.removerUsuario(login)` (remove comunidades das quais o usuário era administrador e retira sua participação das demais) e, por fim, `usuarioService.removerUsuario(id, sessionService)`, que também encerra todas as sessões abertas daquele usuário via `sessionService.fecharSessoesDoUsuario()`. Cada serviço permanece responsável apenas pelo próprio repositório, o que mantém a coesão da camada de serviço mesmo numa operação que atravessa todos os domínios do sistema.

6. CONCLUSÃO

O desenvolvimento das User Stories 05 a 09 e a refatoração de encapsulamento, herança e polimorfismo trouxeram ganhos reais de design: a hierarquia Relacionamento/Mensagem eliminou condicionais de tipo, a Factory centralizou a criação de objetos polimórficos, e o conjunto de 29 exceções específicas tornou o tratamento de erros mais legível e previsível do que o uso de Exception genérica.